

15 — Brain Persistence & Multi-Cloud

Objetivo: garantizar que el **cerebro** de Puglit —todo lo que los agentes aprendieron a lo largo de cientos de builds— **no se pierda** cuando un pod muere ni **se pise** cuando dos pods evolucionan en paralelo. El cerebro vive en un Postgres cloud autoritativo (ACID, seguro) y se respalda con snapshots portables (git/bucket) que se mergean —nunca se sobrescriben— mediante semántica CRDT para datos aditivos y arbitraje por score objetivo para datos mutables. Todo verificado contra `web/lib/brain-sync.ts`, `web/app/api/admin/brain/route.ts`, `web/lib/db.ts`, `web/sql/genetic.sql`, `infra/brain-snapshot.sh` e `infra/brain-restore.sh`.

1. The Brain

El "cerebro" no es una metáfora suelta: es un conjunto concreto de tablas Postgres donde se acumula el **aprendizaje** del swarm. No son los proyectos de los usuarios (eso vive en `puglit_projects` / `puglit_jobs`), sino la inteligencia que mejora build tras build: el diario de los agentes, las métricas de evidencia, los exemplars verificados, las skills evolucionadas por SkillOpt, los módulos cosechados, y el XP/nivel de cada uno de los 75 agentes.

La definición canónica de qué tablas componen el cerebro vive en `web/lib/brain-sync.ts`, partida en **dos clases de datos** que se mergean con reglas distintas:

```
// web/lib/brain-sync.ts
const APPEND: Record<string, { cols: string[]; key: string[]; json: string[] }> = {
  puglit_agent_diary: { ..., key: ["agent_id", "kind", "entry", "created_at"], json: ["embedding"] },
  puglit_metrics: { ..., key: ["name", "value", "created_at"], json: ["meta"] },
  verified_exemplars: { ..., key: ["kind", "code"], json: ["embedding"] },
  puglit_skill_rejects: { ..., key: ["area", "edit", "created_at"], json: [] },
}
const MUTABLE = ["puglit_skills", "puglit_modules"]
// + puglit_agents (solo id, xp, level) → XP arbitrado por MAX
```

1.1 Append-only vs mutable

La distinción es el corazón del diseño. Cada tabla del cerebro cae en una de dos categorías según **cómo se la puede combinar de forma segura** cuando dos copias divergen:

Tabla	Clase	Por qué	Content key (dedup)
<code>puglit_agent_diary</code>	APPEND-ONLY	Cada lección/win/crítica es un hecho histórico inmutable; sumar lecciones de otro pod nunca contradice las propias	<code>(agent_id, kind, entry, created_at)</code>
<code>puglit_metrics</code>	APPEND-ONLY	Cada medición (build/smoke/judge-agreement/ablation) es una observación puntual; más datos = mejor	<code>(name, value, created_at)</code>
<code>verified_exemplars</code>	APPEND-ONLY	Código known-good que pasó el gate; tener más exemplars solo enriquece el retrieval	<code>(kind, code)</code>
<code>puglit_skill_rejects</code>	APPEND-ONLY	Buffer de ediciones de skill rechazadas; el registro de "lo que NO funcionó" solo crece	<code>(area, edit, created_at)</code>
<code>puglit_skills</code>	MUTABLE	Hay UN skill activo por área; dos versiones de la misma área compiten y una gana	arbitrado por <code>val_score</code>
<code>puglit_modules</code>	MUTABLE	Hay UN módulo por <code>name</code> (PK); versiones nuevas reemplazan a viejas	arbitrado por <code>version</code>
<code>puglit_agents</code> (xp, level)	MUTABLE	El XP es un contador monotónico; nunca debe bajar	arbitrado por <code>MAX</code>

La intuición: lo **aditivo** (un diario que crece, métricas que se acumulan) se puede **unir** sin conflicto —es un grow-set—; lo **autoritativo** (cuál skill está activo, qué versión de módulo gana) requiere **decidir un ganador** con un criterio objetivo, no con "el último que escribió gana".

Las definiciones físicas están en `web/sql/genetic.sql`: `puglit_agent_diary` (id BIGSERIAL, embedding JSONB nomic-embed, columnas `outcome / scope` para anti-poisoning), `puglit_metrics`, `verified_exemplars`, `puglit_skills` (`area, version, doc, val_score, status`), `puglit_skill_rejects`, `puglit_modules` (`name` PK, `version, status`), y `puglit_agents` (`xp, level, stats` JSONB).

2. The Durability Problem

El problema que motiva todo este módulo es físico y mundano: **el disco de un pod es efímero**.

Puglit corre en su forma canónica sobre un **RunPod Pod A40 48GB** (ver doc 08). Un Pod es un contenedor: cuando RunPod lo recicla, lo migra de host, o el usuario lo apaga para ahorrar GPU-horas, **todo el filesystem local desaparece**. Si el cerebro viviera solo en un Postgres montado sobre el disco del Pod, cada apagado borraría cientos de builds de aprendizaje acumulado.

Peor aún: el flujo de "100-build brain-training batch" (`infra/train-brain.sh`) está diseñado para pre-cargar el cerebro con apps diversas (juegos retro, calculadoras, verticales de negocio, etc.). Ese cómputo —decenas de horas de GPU— se perdería entero si el pod muere a mitad de camino sin un respaldo durable.

```
RunPod Pod (efímero)
Ollama (qwen2.5-coder:32B) · Next.js web (swarm)
Postgres 14 + pgvector ← el CEREBRO vive acá

△ reciclo/migración/apagado del Pod
→ disco local borrado → cerebro PERDIDO
```

Hay además un segundo riesgo, ortogonal a la durabilidad: la **conurrencia**. Si el usuario corre dos pods (por ejemplo, uno entrenando con `train-brain.sh` y otro sirviendo builds reales), o si el cerebro se restaura desde un snapshot viejo encima de uno más nuevo, un "git push que clobbera" destruiría aprendizaje. La solución tiene que resolver **las dos cosas a la vez**: que no se pierda (durabilidad) y que no se pise (merge sin clobber).

3. Hybrid Architecture

La respuesta de Puglit es deliberadamente **híbrida**: combina lo mejor de dos mundos en vez de elegir uno.

Capa	Rol	Propiedades	Dónde
Postgres cloud	Almacén autoritativo y vivo	ACID, seguro, transaccional, queries en caliente	<code>web/lib/db.ts</code> (<code>Pool</code> , <code>POSTGRES_*</code>)
Snapshots git/bucket	Backup portable	Off-pod, versionado, hand-off entre pods	<code>infra/brain-snapshot.sh</code> / <code>brain-restore.sh</code>

La regla, escrita literalmente en el header de `brain-sync.ts`:

```
"Authoritative live store = the cloud Postgres (POSTGRES_). exportBrain()/mergeBrain() are for
snapshots (git/bucket backup) + bootstrapping/reconciling a fresh or offline pod SAFELY."
```

3.1 Por qué un Postgres cloud autoritativo

El cerebro vivo NO debe estar en el disco efímero del Pod. La conexión en `web/lib/db.ts` apunta por defecto al **pooler de Supabase (Supavisor)** vía TLS:

```
// web/lib/db.ts
_pool = new Pool({
  host: process.env.POSTGRES_HOST, port: parseInt(process.env.POSTGRES_PORT || "5432"),
  database: process.env.POSTGRES_DB || "postgres", user: process.env.POSTGRES_USER,
  password: process.env.POSTGRES_PASSWORD, max: parseInt(process.env.POSTGRES_POOL_MAX || "5"),
  // Supabase requires TLS. Set POSTGRES_SSL=disable for a local plain DB.
  ssl: process.env.POSTGRES_SSL === "disable" ? undefined : { rejectUnauthorized: false },
})
```

Apuntando `POSTGRES_HOST` a un Supabase/Neon gestionado, el cerebro sobrevive a cualquier reciclo del Pod: el Pod se vuelve **stateless** respecto al aprendizaje. ACID nos da la garantía que un filesystem no da:

cada `INSERT / UPDATE` del merge es atómico y aislado.

3.2 Por qué snapshots portables además del cloud

Postgres autoritativo es necesario pero no suficiente:

- **Belt-and-suspenders.** Un drop accidental, una migración rota, o un proveedor caído no deben ser un single point of failure. El snapshot es la copia off-store.
- **Portabilidad / hand-off.** Un snapshot JSON es trivialmente movable entre nubes: git push a un repo `puglit-brain`, o un bucket. Bootstrappar un pod nuevo es `psql genetic.sql + brain-restore.sh`.
- **Versionado.** Cada snapshot queda en git con timestamp (`snapshots/brain-<TS>.json`), permitiendo auditar cómo evolucionó el cerebro y revertir si un batch lo degradó.



El endpoint que conecta ambas capas es `web/app/api/admin/brain/route.ts`: `GET` exporta, `POST` mergea. Está protegido por `isServiceRequest` (header `x-puglit-service: $PUGLIT_SERVICE_TOKEN`) y tiene `maxDuration = 800` para tolerar merges grandes.

4. Merge Semantics

El merge (`mergeBrain` en `brain-sync.ts`) NO es un overwrite. Procesa cada clase de dato con su propia regla. Esta es la tabla maestra **clase de dato** → **regla de merge**:

Clase de dato	Tabla(s)	Regla de merge	Resultado
APPEND-ONLY	diary, metrics, exemplars, skill_rejects	UNION por content key (CRDT grow-set)	Se insertan solo las filas cuya key no existe; nada se sobrescribe
MUTABLE: skills	<code>puglit_skills</code>	Unir versiones como <code>archived</code> → consolidar active = mayor <code>val_score</code> held-out	Cada área queda con su mejor skill validado activo
MUTABLE: modules	<code>puglit_modules</code>	<code>UPSERT ... WHERE EXCLUDED.version > current.version</code>	Gana la versión más alta, objetivamente
MUTABLE: XP	<code>puglit_agents</code>	<code>xp = GREATEST(current, incoming)</code> , igual con <code>level</code>	Nunca se pierden niveles

4.1 APPEND-ONLY → unión por content key (CRDT grow-set)

Para las cuatro tablas aditivas, `unionInsert` recorre las filas entrantes y solo inserta las que **aún no existen** según su content key. La comparación usa `IS NOT DISTINCT FROM` (NULL-safe), así dos pods que generaron el mismo diary entry no lo duplican:

```
// web/lib/brain-sync.ts - unionInsert
const where = def.key.map((k, i) => `${k} IS NOT DISTINCT FROM ${i + 1}`).join(" AND ")
const exists = (await query(`SELECT 1 FROM ${table} WHERE ${where} LIMIT 1`, ...)).rows.length
if (exists) continue // ya está → no duplicar
// si no → INSERT (JSONB cols como ::jsonb)
```

Esto es exactamente un **CRDT grow-set (G-Set)**: una estructura donde la única operación es "agregar", la unión de dos réplicas es su unión de conjuntos, y el resultado es **conmutativo, asociativo e idempotente**. No importa el orden en que dos pods se mergeen entre sí ni cuántas veces: convergen al mismo conjunto sin conflicto. Por eso el diario, las métricas y los exemplars pueden crecer en paralelo en N pods y reconciliarse sin perder ni un dato.

4.2 MUTABLE → arbitraje por score objetivo

Las tablas mutables NO pueden ser grow-sets: hay **un** estado canónico (el skill activo por área, la versión vigente de un módulo, el XP de un agente). Acá last-write-wins sería un desastre —un pod que terminó tarde con un skill peor pisaría a uno mejor—. La regla es **arbitrar por un criterio objetivo**, no por timestamp:

- **Skills** → el ganador es el de **mayor val_score** (score sobre un set held-out de validación de SkillOpt). Primero el merge archiva todas las versiones entrantes, luego `consolidateActiveSkills` elige la mejor por área.
- **Modules** → gana la **mayor versión** (el `UPSERT` solo actualiza `WHERE EXCLUDED.version > puglit_modules.version`).
- **XP** → `GREATEST(actual, entrante)`: monótonico, nunca decrece.

El comentario del código lo dice sin ambigüedad: "*arbitrated by an OBJECTIVE score, never last-write-wins ... → no incongruent learnings*." El objetivo último es evitar **aprendizajes incongruentes**: que el cerebro nunca contenga un estado "ganador" que en realidad rinde peor que el que pisó.

5. Conflict Resolution

Cuando dos snapshots (o un snapshot y el cerebro vivo) divergen sobre el mismo objeto mutable, el conflicto se resuelve con un criterio distinto por tipo. Ninguno usa la hora de escritura.

5.1 Skills: mayor val_score held-out

`puglit_skills` guarda múltiples versiones por área, cada una con su `val_score` (medido contra 5 tareas frozen → rollout de blueprint → `objectiveScore`, ver doc 03). Al mergear, todas las versiones entrantes se insertan como `archived` (preservando el historial), y luego `consolidateActiveSkills` elige UNA activa por área:

```
// web/lib/brain-sync.ts - consolidateActiveSkills
await query("UPDATE puglit_skills SET status='archived' WHERE status='active'")
await query(`UPDATE puglit_skills s SET status='active'
  FROM (SELECT DISTINCT ON (area) area, id FROM puglit_skills
    ORDER BY area, val_score DESC NULLS LAST, version DESC) best
  WHERE s.id = best.id`)
```

`DISTINCT ON (area) ... ORDER BY val_score DESC` selecciona, por cada área, la versión con **el mejor score held-out**; empate de score se desempata por versión mayor. El skill activo es siempre el objetivamente mejor, sin importar qué pod lo produjo ni cuándo.

5.2 Modules: mayor versión

Un módulo es único por `name` (PK). El conflicto se resuelve quedándose con la **versión más alta**, vía un `UPSERT` condicional:

```
// web/lib/brain-sync.ts - merge de puglit_modules
INSERT INTO puglit_modules (name, ...) VALUES (...)
ON CONFLICT (name) DO UPDATE SET version=EXCLUDED.version, status=EXCLUDED.status, ...
WHERE EXCLUDED.version > puglit_modules.version
```

Si el módulo entrante tiene versión menor o igual, el `WHERE` lo descarta: el local más nuevo se queda. Si es mayor, reemplaza. Determinista, sin reloj.

5.3 XP / level: MAX

El progreso de cada agente es monótonico. El merge nunca degrada un agente:

```
// web/lib/brain-sync.ts - merge de puglit_agents
UPDATE puglit_agents SET xp=GREATEST(COALESCE(xp,0),$2),
  level=GREATEST(COALESCE(level,1),$3) WHERE id=$1
```

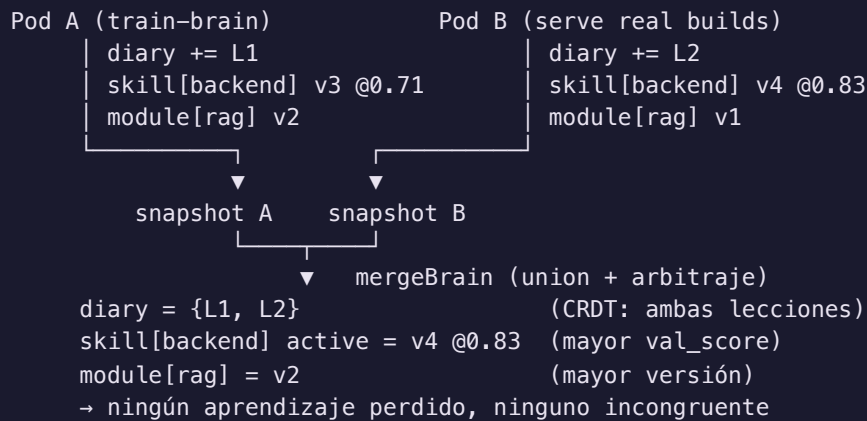
Dos pods que jugaron torneos distintos suman XP por separado; el merge se queda con el máximo de cada uno. (Nota de honestidad: como cada lado arbitra por MAX en vez de sumar, dos historias paralelas no se *acumulan*, se *consolidan* al mayor —es la decisión correcta para no perder niveles sin inflar XP por doble conteo.)

6. No-Clobber Guarantee

La propiedad central que emerge de las secciones 4 y 5 es la **No-Clobber Guarantee**: *múltiples agentes y múltiples pods pueden evolucionar el cerebro en simultáneo sin pisarse*. Se sostiene en tres patas:

1. **Aditivo es conmutativo.** Las tablas APPEND-ONLY son grow-sets CRDT: la unión es orden-independiente e idempotente. N pods escribiendo diario/métricas/exemplars en paralelo convergen al mismo superconjunto. Re-mergear el mismo snapshot dos veces no cambia nada (la key ya existe → `continue`).

2. **Mutable es arbitrado, no temporal.** Skills/módulos/XP se deciden por score/versión/máximo — criterios objetivos y conmutativos—, no por "quién escribió último". Un pod lento con un skill peor **no puede** pisar a uno con mejor `val_score`.
3. **Tolerancia a fallos por fila.** Cada operación del merge está envuelta en `.catch(() => ...)`: una fila que falle (tabla inexistente en un pod recién bootstrapeado, conflicto de tipo) no aborta el merge entero. El export es igualmente defensivo (`try { ... } catch { snap[t] = [] }`), así un cerebro parcial se exporta igual.



El resultado: la convergencia es **independiente del orden de merge**. No hace falta coordinación, locks distribuidos, ni un "líder". Esa es la promesa que hace operable el multi-pod.

7. Snapshot & Restore

Dos scripts de `infra/` implementan el flujo de durabilidad sobre el endpoint `/api/admin/brain`.

7.1 `brain-snapshot.sh` — export

Corre en cron o después de cada batch. Hace `GET /api/admin/brain` con el service token, guarda el JSON, y lo commitea a un repo git `puglit-brain` (o, si no hay repo configurado, lo sube a un bucket temporal para tener al menos una copia off-pod):

```
# infra/brain-snapshot.sh (esencial)
curl -s -H "x-puglit-service: $SVC" "$BASE/api/admin/brain" -o "$OUT"
if [ -n "${BRAIN_REPO:-}" ] && [ -d "$BRAIN_REPO/.git" ]; then
  cp "$OUT" "$BRAIN_REPO/latest.json"
  cp "$OUT" "$BRAIN_REPO/snapshots/brain-${TS}.json"
  ( cd "$BRAIN_REPO" && git add -A && git commit -q -m "brain snapshot ${TS}" && git push -q )
else
  # sin repo → bucket temporal (off-pod copy mínima)
  URL=$(curl -s -F "file=@$OUT" https://tmpfiles.org/api/v1/upload | sed -E '...')
fi
```

Guarda dos copias: `latest.json` (el puntero que usa restore) y `snapshots/brain-<TS>.json` (historial versionado). El push a git es la copia durable canónica.

7.2 brain-restore.sh — merge resumable

Corre al arrancar un pod (después de `psql genetic.sql`). Trae el snapshot (de un path, una URL, o `BRAIN_REPO/latest.json`) y lo mergea con `POST /api/admin/brain`:

```
# infra/brain-restore.sh (esencial)
SRC="${SNAPSHOT:-${BRAIN_REPO:-}/latest.json}"
case "$SRC" in http*) curl -s "$SRC" -o "$TMP" ;; *) cp "$SRC" "$TMP" ;; esac
[ -s "$TMP" ] || { echo "empty/missing snapshot"; exit 1; }
curl -s -X POST -H "x-puglit-service: $SVC" -H "Content-Type: application/json" \
  --data-binary "@${TMP}" "$BASE/api/admin/brain" | python3 -m json.tool
```

Resumable / safe-to-rerun por construcción. El header del script lo promete: *"Safe to run anytime: it UNIONS additive learnings and arbitrates skills by validation score — it never clobbers a newer brain."* Por la No-Clobber Guarantee (sección 6), correr restore N veces, o sobre un cerebro ya poblado, o a mitad de un batch interrumpido, es seguro: las filas ya presentes se saltean y los mutables se arbitran. Si `train-brain.sh` se corta en el build 60 de 100, un nuevo pod restaura los 60 ya snapshotados y sigue —no reemplaza de cero ni duplica.

El endpoint también admite `POST /api/admin/brain?consolidate=1`, que **solo** re-ejecuta `consolidateActiveSkills()` sin mergear nada —útil para re-arbitrar el skill activo tras una corrida de SkillOpt sin un round-trip de snapshot.

```
Pod start
| psql web/sql/genetic.sql          (crea/asegura tablas — idempotente)
| brain-restore.sh                  (GET latest.json → POST merge)
▼
swarm corriendo con el cerebro restaurado
| ...builds, SkillOpt, XP...
| brain-snapshot.sh (cron / post-batch) → git push puglit-brain
▼
Pod muere → el cerebro vive en Postgres cloud + git
```

8. Operational Playbook

8.1 Separar el cerebro a un Postgres gestionado (Supabase / Neon)

El paso clave para hacer el Pod stateless es **mover el almacén autoritativo fuera del Pod**. Como `web/lib/db.ts` lee toda la conexión de `POSTGRES_*` y ya habla TLS contra el pooler de Supabase por defecto, la migración es de configuración, no de código:

1. **Provisionar** un Postgres gestionado con `pgvector` (Supabase o Neon). El cerebro usa JSONB para embeddings; `pgvector` no es estrictamente requerido por estas tablas pero sí por los módulos `rag` /retrieval de la plataforma.
2. **Aplicar el schema:** `psql "$POSTGRES_URL" -f web/sql/genetic.sql` (idempotente: todo es `CREATE TABLE IF NOT EXISTS` + `ALTER ... ADD COLUMN IF NOT EXISTS`).
3. **Configurar el Pod** con `POSTGRES_HOST/PORT/DB/USER/PASSWORD` apuntando al gestionado. Con Supabase, usar el **pooler Supavisor** (`aws-1-<region>:5432`), no el host directo —el host directo de

Supabase puede no tener IPv4 y un Pod no llegaría.

4. **Dejar SSL por defecto** (`rejectUnauthorized:false`); solo para un Postgres local plano se setea `POSTGRES_SSL=disable` .
5. **Sembrar** el roster con `POST /api/genetic/seed` si es un cerebro fresco, luego `brain-restore.sh` para traer el último snapshot.

Con esto, reciclar/migrar el Pod no toca el cerebro: el aprendizaje vive en el gestionado y los snapshots git son el respaldo.

8.2 Cron de durabilidad

Programar `brain-snapshot.sh` en cron (post-batch y/o cada N horas) con `SVC=$PUGLIT_SERVICE_TOKEN` , `BASE=http://localhost:3000` y `BRAIN_REPO=/path/to/puglit-brain` . Así cada snapshot queda versionado en git, off-pod, recuperable.

8.3 Bootstrap de un pod nuevo / disaster recovery

Orden canónico al arrancar un pod (parte de `setup-gpu-box.sh`):

```
psql ... -f web/sql/genetic.sql      # 1. schema (idempotente)
POST /api/genetic/seed               # 2. roster de 75 agentes (si fresco)
brain-restore.sh                     # 3. merge del último snapshot (safe-to-rerun)
```

Si el Postgres gestionado se perdió por completo, este mismo flujo reconstruye el cerebro desde el último `latest.json` de git. La pérdida máxima es lo aprendido entre el último snapshot y la caída —acotable bajando el intervalo del cron.

8.4 Apps generadas: storage propio, separado del cerebro

Importante no confundir dos cosas distintas. El cerebro (este doc) es estado **de la plataforma Puglit**. Las **apps generadas** que se entregan al usuario traen su **propio** spine de DB (`spine/` , `export const pool = new Pool()`) y corren contra **su propio** Postgres —típicamente local/efímero durante el build (el patrón "Postgres local efímero" del runtime gate de `build-local.mjs`), o el Postgres del usuario en su deploy BYO (GitHub + Vercel, `infra/deploy.sh`). Esas DBs no comparten nada con el cerebro: `puglit_projects / puglit_jobs` guardan la metadata del build en el cerebro, pero las tablas de negocio de la app generada viven aparte, en la cuenta del usuario. Separar el cerebro a un gestionado y manejar las apps generadas localmente son decisiones operativas **independientes**.

8.5 Checklist operativo

Acción	Comando / endpoint	Frecuencia
Snapshot durable	<code>brain-snapshot.sh</code> → git push	cron + post-batch
Restore / bootstrap	<code>brain-restore.sh</code> (merge)	al arrancar pod
Re-arbitrar skills	<code>POST /api/admin/brain?consolidate=1</code>	tras SkillOpt
Export manual	<code>GET /api/admin/brain</code>	ad-hoc / debug
Schema fresco	<code>psql -f web/sql/genetic.sql</code>	bootstrap (idempotente)

Cierre

El cerebro de Puglit es híbrido por diseño: **Postgres cloud autoritativo** (ACID, seguro, vivo) como fuente de verdad, y **snapshots git/bucket portables** como respaldo durable y vehículo de hand-off. El merge nunca clobbera: lo aditivo se une como **CRDT grow-set** y lo mutable se arbitra por **score objetivo** (`val_score` held-out para skills, versión para módulos, MAX para XP). De ahí sale la **No-Clobber Guarantee** —multi-agente y multi-pod sin pisarse, con convergencia independiente del orden de merge. El resultado operativo: un Pop efímero puede morir, migrarse o multiplicarse, y el aprendizaje acumulado **ni se pierde ni se contradice**.